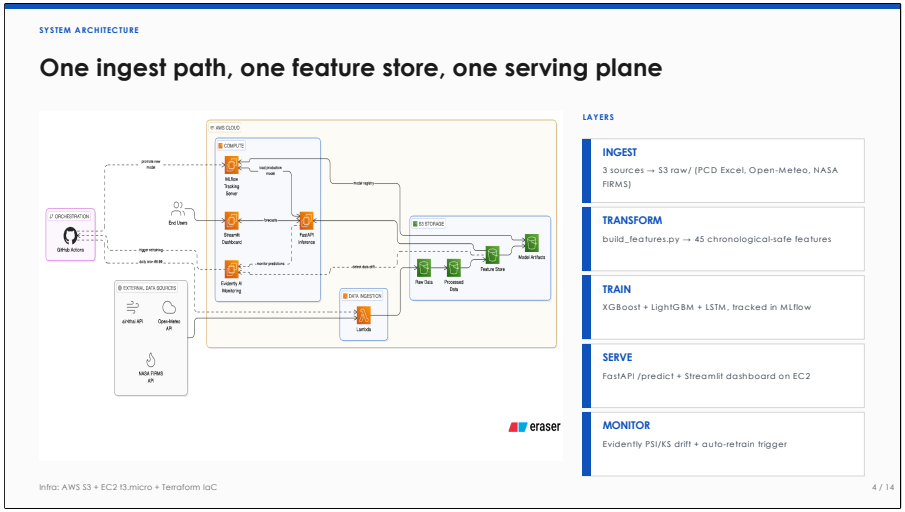
Good morning Today we are presenting the **ChiangMai Air Quality Alert System** — or CAAS — our end-to-end MLOps project for the Data En

Every year, from February to April, Chiang Mai suffers a PM2.5 air quality crisis.

In the 2024 dry season alone, PM2.5 peaked at **241 micrograms per cubic meter** [point]. That is **sixteen times** the WHO 24-hour safe limit of 15.
This affects **1.2 million people** in the metro area. Public health data from the Thai Pollution Control Department shows a **12 percent increase in all-cause mortality** on high-PM days.
And here's the gap: the existing public tool, air4thai.pcd.go.th, only shows **current** levels. It doesn't forecast. If you are an asthmatic parent, a school administrator, or a tourism operator, you find out the air is dangerous when it already is — too late to plan.
CAAS closes that gap by forecasting 1, 3, and 7 days ahead.

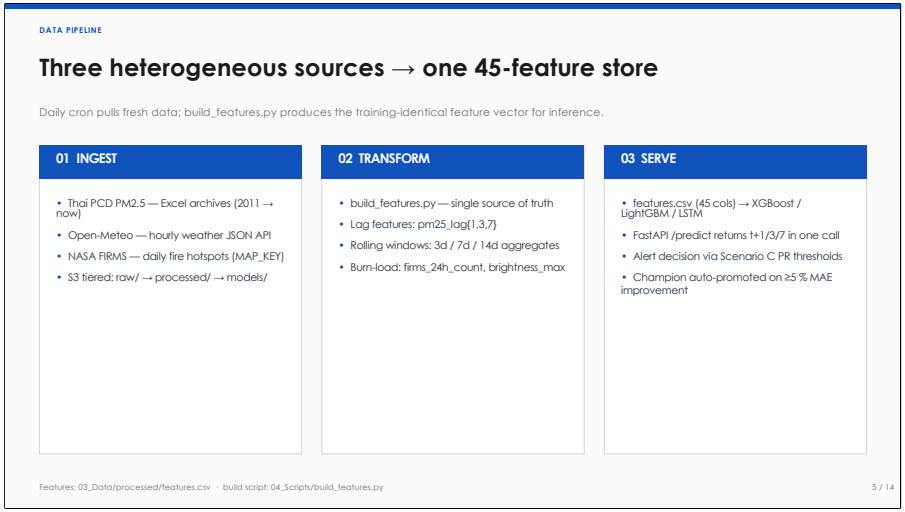
We set three objectives.

First — forecast. Daily PM2.5 at t+1, t+3, t+7, with a strict leakage-free chronological validation split.
Second — alert. Classify each horizon against the 50 microgram hazard threshold, with operationally useful precision and recall.
Third — MLOps. Deploy the full stack — ingest, train, serve, monitor — with automated drift detection and champion-challenger promotion.
Our contribution is **integration**, not novelty. A multi-horizon forecast, with fire-feature ablation, running end-to-end on a low-budget EC2 footprint.

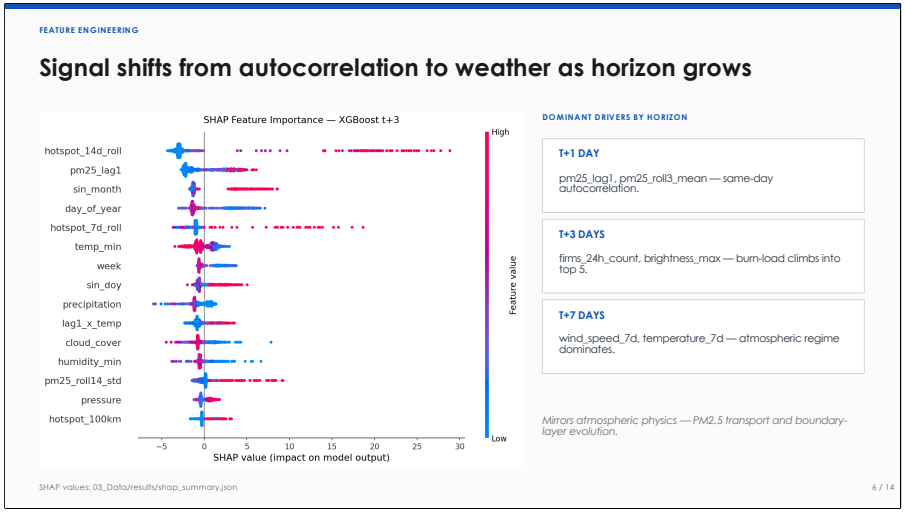


The architecture has **five layers** [point down the column]. // **Ingest** pulls from three sources into S3 raw. // **Transform** runs build_features.py and produces 45 chronological-safe features. // **Train** fits three models — XGBoost, LightGBM, and LSTM — all tracked in MLflow. // **Serve** exposes a FastAPI prediction endpoint and a Streamlit dashboard on an EC2 instance. // **Monitor** runs Evidently-based PSI and KS drift checks with an auto-retrain trigger.

Everything runs on AWS — S3, an EC2 t3.small — and everything is provisioned as code via Terraform.



Three heterogeneous sources, one feature store.
Ingest [point]: Thai PCD PM2.5 from Excel archives going back to 2011 — plus live daily readings via the air4thai API. Open-Meteo for 11 daily weather variables. NASA FIRMS for satellite-detected fire hotspots within 50 and 100 kilometers. Everything lands in S3 tiered as raw, processed, and models.
Transform: build_features.py is our single source of truth. It produces lag features, 3/7/14-day rolling windows, and burn-load features like firms_24h_count and brightness_max. Crucially, the **same script** builds features for both training and inference — that guarantees no train-serve skew.
Serve: the final features.csv feeds XGBoost, LightGBM, and LSTM. FastAPI returns all three horizons in one call. The alert decision uses our Scenario C PR-tuned thresholds, not a fixed value. And the champion is auto-promoted on a 5 percent MAE improvement gate.



Our SHAP analysis reveals how the model's reasoning **shifts** with horizon.
At t+1, the dominant drivers are pm25_lag1 and the 3-day rolling mean. Same-day autocorrelation carries almost all the signal.
At t+3, burn-load features like firms_24h_count and brightness_max climb into the top five. Two- to three-day fire smoke transport becomes predictive.
At t+7, the weather regime takes over. wind_speed_7d and temperature_7d dominate. At a week out, you're effectively forecasting the weather pattern.
This mirrors atmospheric physics — boundary-layer evolution and transport — which is an honesty check that the model is learning something real, not memorizing noise.

Our original proposal had two models — XGBoost and LSTM. During training we added **LightGBM** as a third candidate, and it became our cha

LightGBM's test-set MAE is **5.12 at t+1, 6.77 at t+3, 8.25 at t+7** [point to headline numbers]. That beats XGBoost on every horizon — 5.31, 7.08, 8.90. And it beats LSTM clearly — 6.67, 8.87, 8.06.

Why LightGBM won:

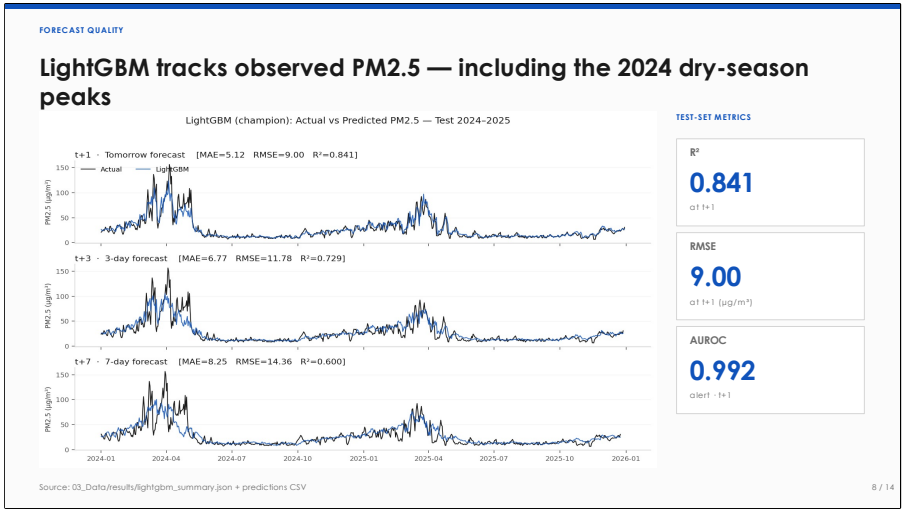
Lower MAE at every horizon — 3.6 percent better than XGBoost at t+1

Best alert-F1 at t+1 at 0.852 — operationally usable

Same 45-feature input — a fair head-to-head, not a feature-set cheat

Passed our 5 percent MAE improvement gate on April 18th, so the champion promotion is auditable in MLflow

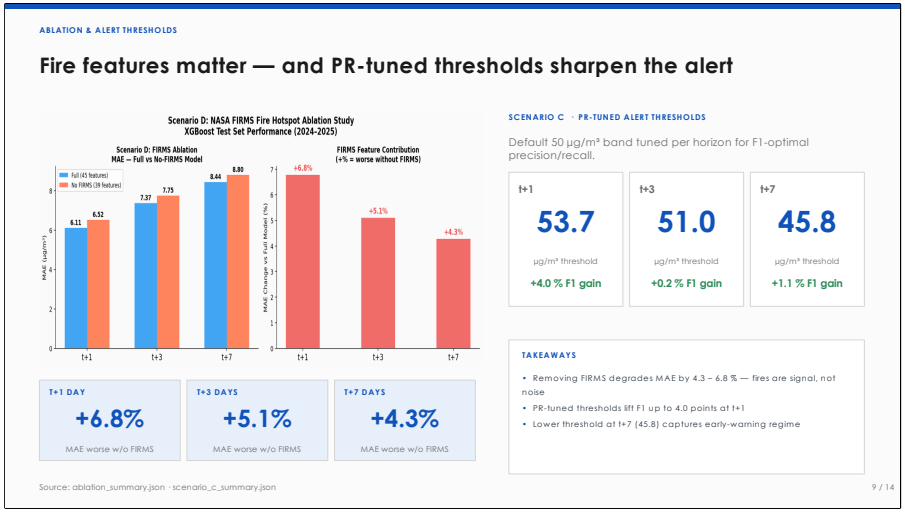
The takeaway: on this dataset size, **tabular models with good features beat sequence models**. The LSTM only wins t+7 R^2 by a thin margin.



This is the LightGBM champion on the two-year test set — 2024 and 2025 [point to chart]. The blue line is our prediction; the black line is the observed PM2.5.

Notice the model tracks the **2024 dry-season peak** where values exceeded 200 micrograms. We don't just predict the easy days — we capture the crisis events, which is what operationally matters.

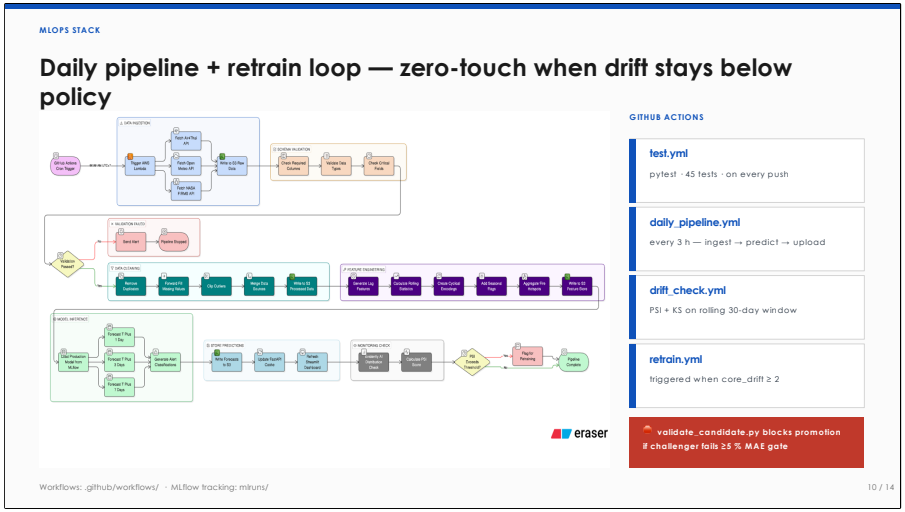
The headline test-set numbers: **R^2 of 0.841** at t+1, **RMSE of 9.00** micrograms, and an **alert AUROC of 0.992** [point]. That last number means the model very cleanly separates alert from non-alert days at t+1.



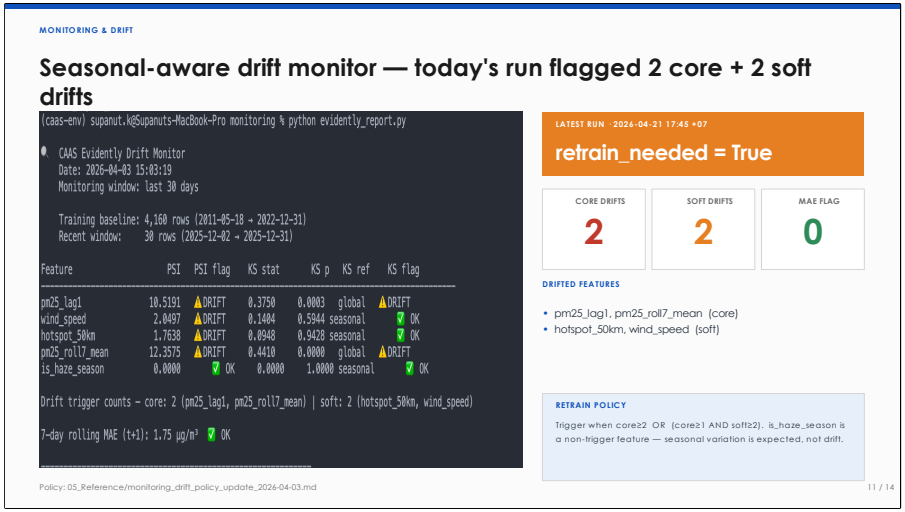
First — an **ablation study**. We retrained LightGBM without the NASA FIRMS fire features. The MAE **degraded by 6.8 percent at t+1, 5.1 percent at t+3, and 4.3 percent at t+7** [point to the red numbers]. So the fires are real signal, not noise. That's important because adding a remote-sensing source has to pay for itself — and here it does.

Second — **Scenario C**. Instead of a fixed 50-microgram alert threshold, we computed the F1-optimal threshold per horizon using PR curves. The tuned thresholds are **53.7 at t+1, 51.0 at t+3, and 45.8 at t+7** [point]. The shorter horizons can afford a higher threshold because confidence is tight. The longer horizon uses a *lower* threshold to catch early-warning regime — at t+7 we want to lean toward recall because missing a hazard week is worse than a false alarm.

F1 gains up to 4 points at t+1 — real, measurable improvement over the fixed threshold.



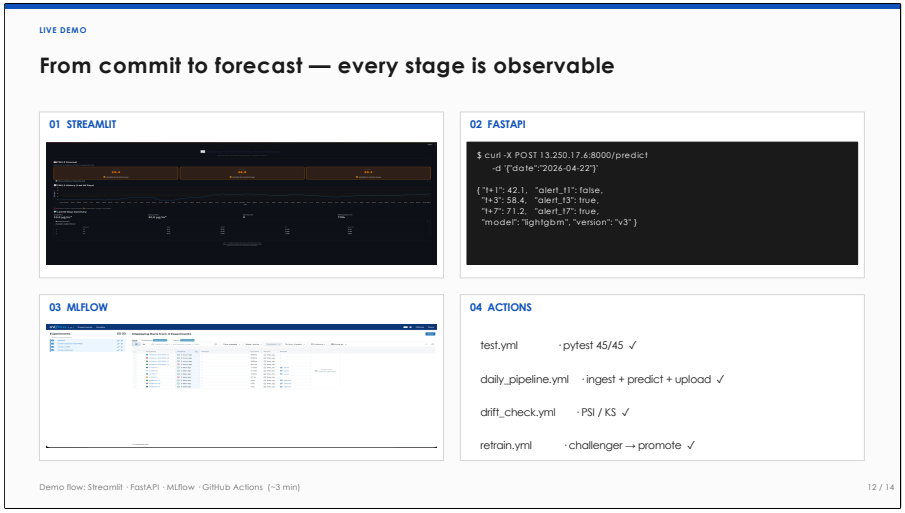
Everything is automated through **four GitHub Actions workflows** [point down the list].
test.yml runs pytest on every push — 45 tests across the API, feature engineering, and models. That's our quality gate.
daily_pipeline.yml runs every three hours. Ingests fresh data, rebuilds features, runs inference, uploads results to S3. Three minutes per run.
drift_check.yml runs PSI and KS tests on a rolling 30-day window.
And retrain.yml fires when drift exceeds policy. It trains a challenger model, and validate_candidate.py **blocks promotion** if the challenger fails the 5 percent MAE gate or the alert-F1 floor.
So the retrain loop is **zero-touch when everything's healthy** — and **safe when it's not**, because a bad challenger cannot replace the champion automatically.



This is the output from our actual drift monitor, run on April 21 [point to timestamp]. // It flags **retrain_needed = True** — two core feature drifts on pm25_lag1 and pm25_roll7_mean, and two soft drifts on hotspot_50km and wind_speed.

Our retrain policy is: trigger when **core ≥ 2**, OR **core ≥ 1 AND soft ≥ 2**. Today's run hits the first condition — so retrain is scheduled.

One design choice worth mentioning: **is_haze_season is explicitly a non-trigger feature**. Seasonal variation is expected — we don't want to retrain every February just because it's haze season again. That's the difference between drift and seasonality, and we encoded that in the policy document.



I'll walk through four stages of our system live. [Switch to browser / terminal]
(1) Streamlit dashboard at 13.250.17.6:8502 — this is what a Chiang Mai resident would actually see. Today's PM2.5, alert banner, the t+1/t+3/t+7 forecast cards, and the 60-day history chart. [Show dashboard]
(2) FastAPI — I'll hit the same endpoint programmatically. [Run curl] curl 13.250.17.6:8000/forecast. You get back the three horizons, alert flags per horizon, and the model version — lightgbm v3. This is what any downstream consumer — an SMS gateway, a partner app — would integrate with.
(3) MLflow at 13.250.17.6:5001. Every experiment is tracked — all three models, all three horizons, with metrics, hyperparameters, and artifacts. This is the audit trail for our champion promotion decision. [Show MLflow runs list]
(4) GitHub Actions. [Switch to repo] All four workflows green. The retrain workflow is on standby waiting for the next scheduled run.
Commit to forecast — every stage is observable.

COST & SCALABILITY

End-to-end MLOps stack for about $0.55 per day

DAILY COST BREAKDOWN (AWS ap-southeast-1)

COMPONENT	USD / DAY	NOTES
EC2 t3.micro (24 h)	$0.24	Host FastAPI + Streamlit + MLflow
S3 storage (~1 GB)	$0.03	raw/processed/models tiers
S3 requests + egress	$0.05	Daily inference pulls
GitHub Actions minutes	$0.18	4 workflows, mostly cached
NASA FIRMS + Open-Meteo	$0.00	Free tiers, MAP_KEY only
TOTAL	$0.55	~$16.50 / month

SCALABILITY PATH

SPOT INSTANCE
EC2 spot can cut compute by ~70 % for retrain only

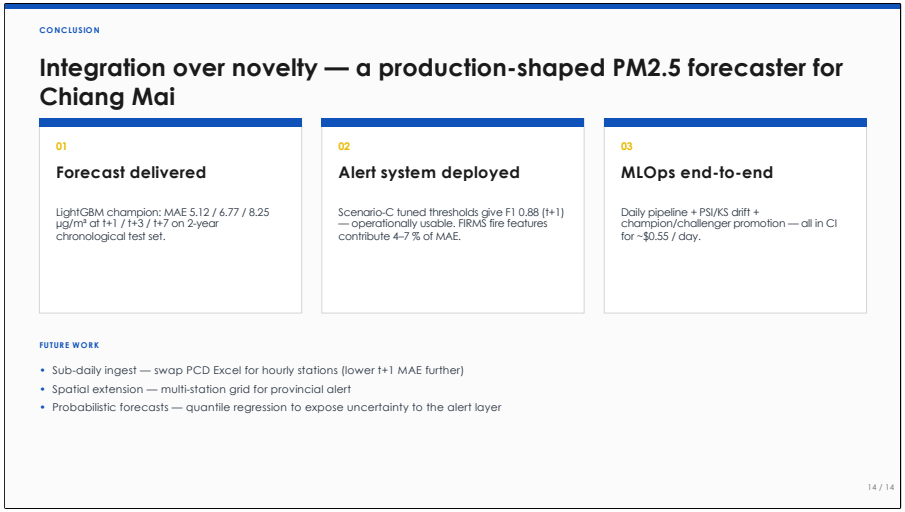
LAMBDA INFERENCE
Move /predict to Lambda when QPS > 10 k/day

SCHEDULED SHUTDOWN
Only run MLflow UI on demand (dev credentials)

MULTI-REGION
Replicate to BKK region for sub-50 ms p99 latency

Pricing: AWS ap-southeast-1, on-demand, April 2026

The total daily cost is **fifty-five cents** — about **$16.50 a month** [point].
Breakdown: EC2 t3.small at 24 cents, S3 storage at 3 cents, S3 requests and egress at 5 cents, GitHub Actions minutes at 18 cents. NASA FIRMS and Open-Meteo are free-tier.
To address the professor's feedback on cost realism — this is the **always-on serving cost**. If we wanted to drop this further, we have three clear paths:
Spot instances for retraining only — 70 percent discount on compute
Lambda inference if daily QPS exceeds 10,000 — pay per request instead of per hour
Scheduled shutdown of the MLflow UI when not in development
For the target user — a municipality or a public-health dashboard — fifty-five cents a day is well within any operational budget.



Our thesis framing: **integration over novelty**.
Forecast delivered — LightGBM champion with MAE 5.12, 6.77, 8.25 on a strict two-year chronological test set.
Alert system deployed — Scenario-C tuned thresholds give an F1 of 0.88 at t+1. FIRMS fire features contribute 4 to 7 percent of MAE.
MLOps end-to-end — daily pipeline, PSI/KS drift, champion-challenger promotion — all running in CI for about fifty-five cents per day.
Future work: sub-daily ingest to lower t+1 MAE further, spatial extension to a multi-station grid, and probabilistic forecasts with quantile regression so the alert layer can expose uncertainty.
Thank you. We're happy to take questions.